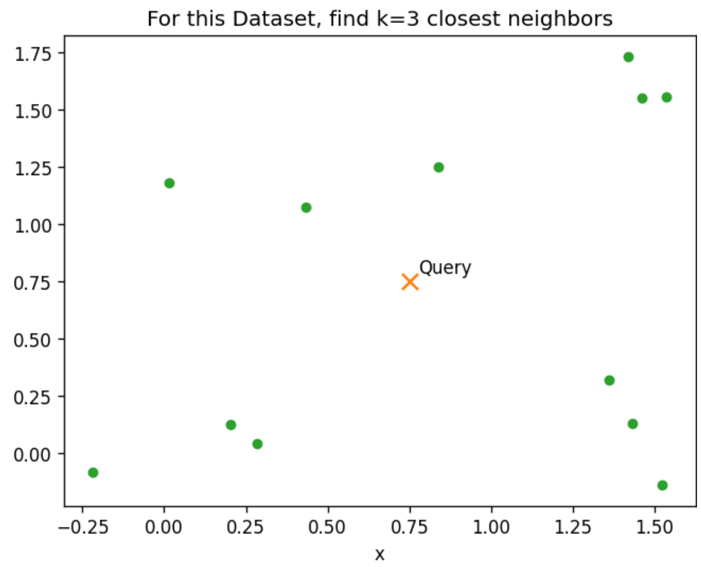
Similarity Searchin Vector Databases:
kNN, IVF, HNSW

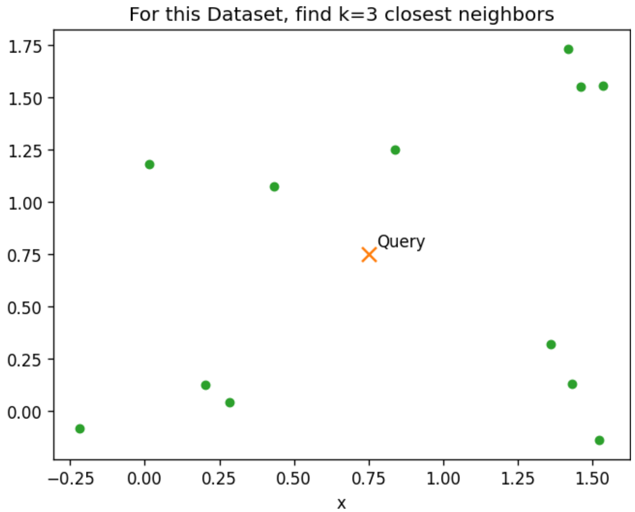
An overview

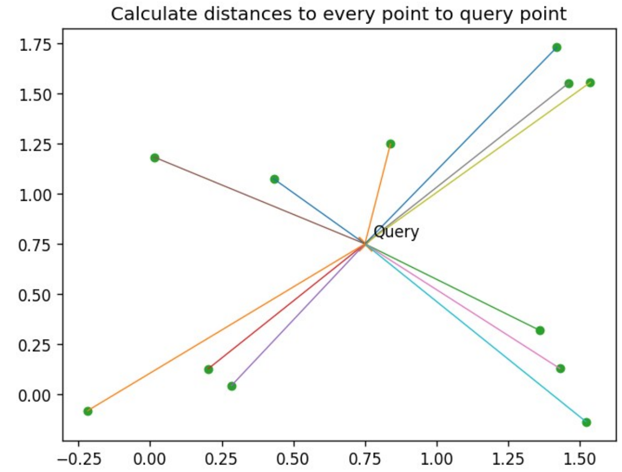
K Nearest Neighbors (kNN)

- Finds the k closest points to a query
- By computing distances to all n points
- Cost: ~O(n) per query → slow at scale



kNN- algorithm





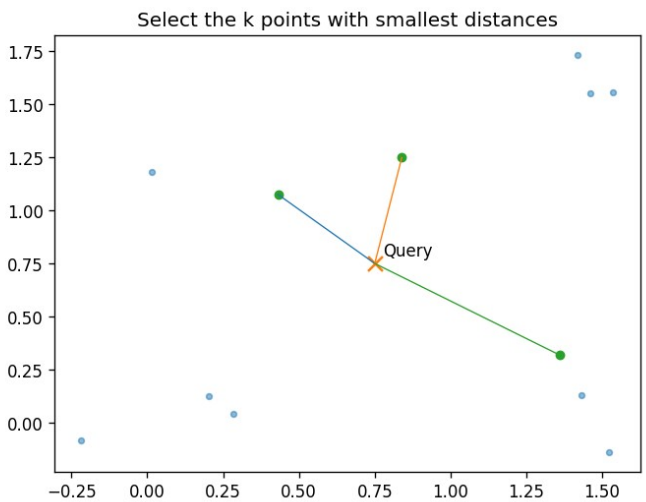
Distance
1.275413
0.847395
0.829449
0.746751
0.921099
1.174519
1.072903
1.187165
1.125681
0.454620
0.853182
0.505817

Distances

Distance
0.454620
0.505817
0.746751
0.829449
0.847395
0.853182
0.921099
1.072903
1.125681
1.174519
1.187165
1.275413

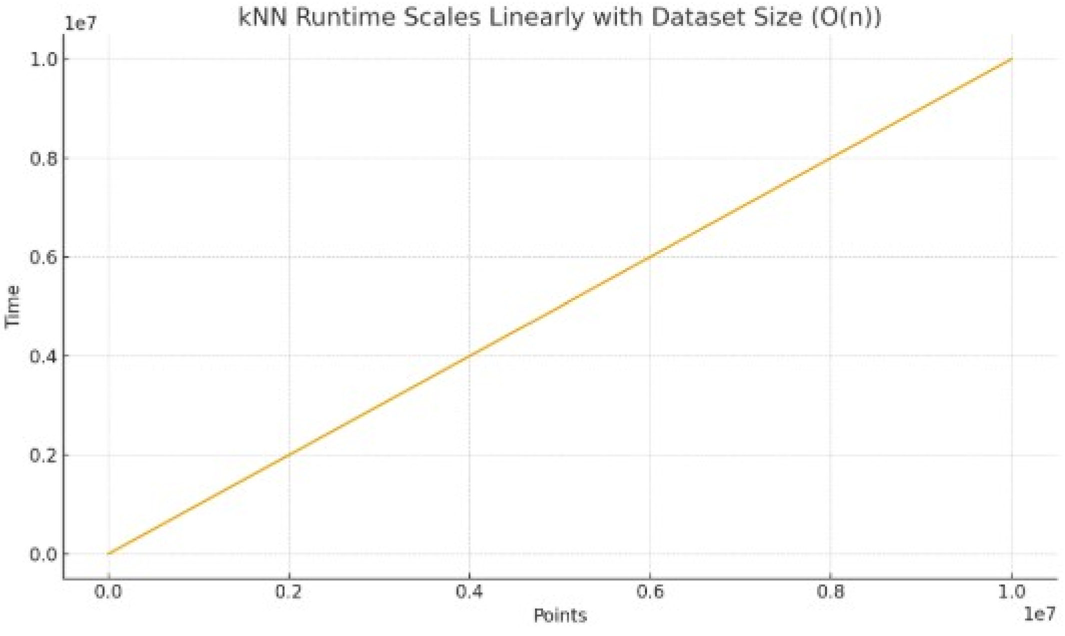
Sorted Distances

Select k closest



Why kNN Doesn’t Scale

- Per-query time grows linearly with data size: exact kNN must compare a query to every point → O(n) work per query
- latency explodes as n grows.

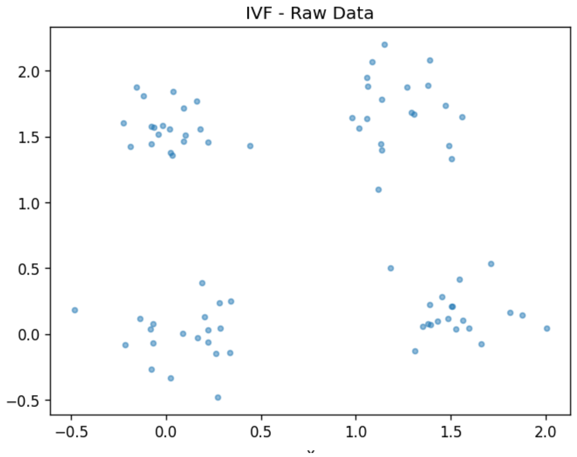


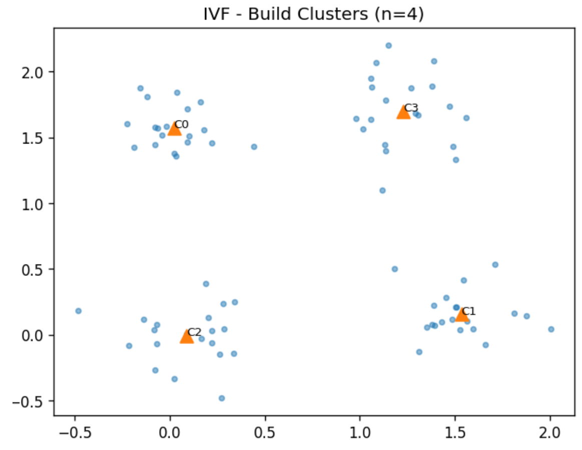
IVF: Inverted File Index — Idea

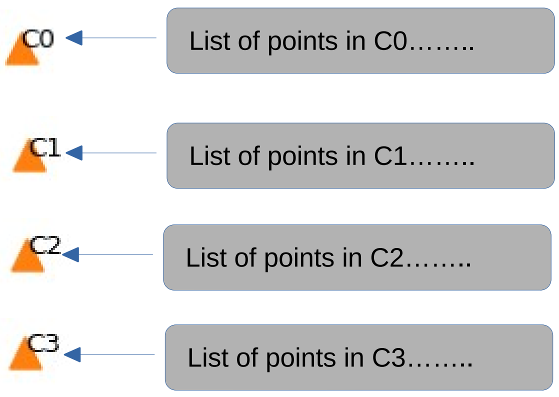
- Partition space into n clusters
  - Rule of thumb: 1000 clusters for 10000000 points -
- At query time probe only k relevant clusters

IVF: Build Clusters

Partition space into n_lists=4 clusters, build list of points in each cluster







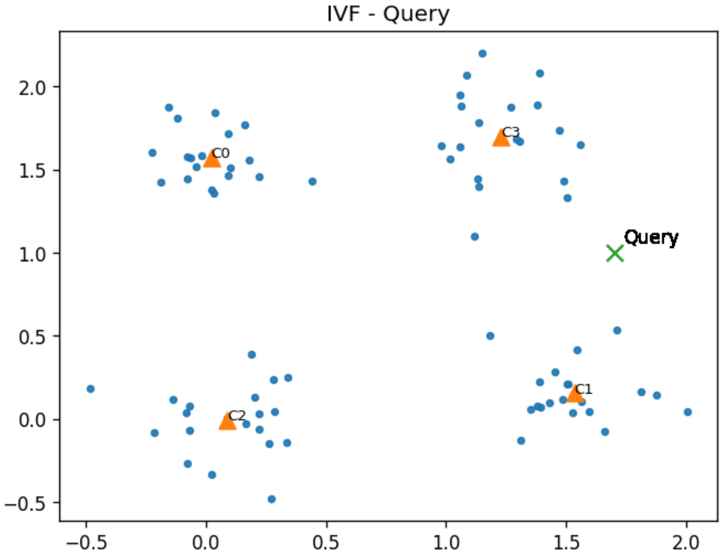
This is an iterative process (KMeans)

1) Randomly place 4 centroids
2) Assign each point to its closest cluster
3) Move centroid to center of its points
4) Go to 2 until no points change cluster

Have list of all points in each cluster

IVF: Query time

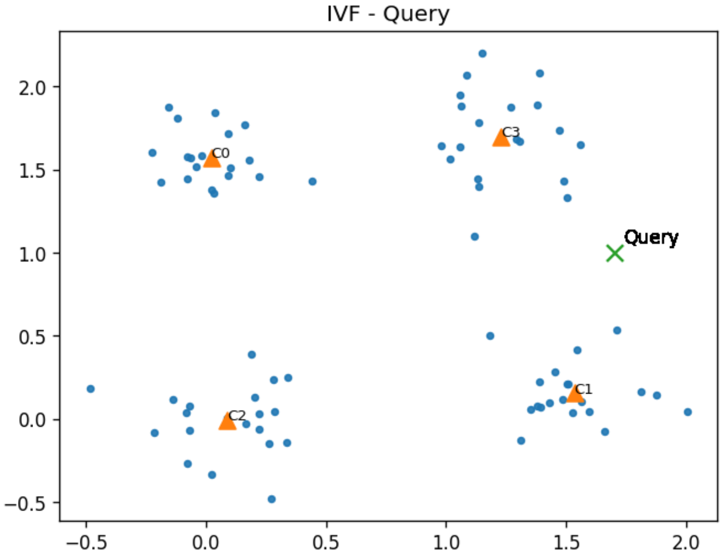
Ex. Find closest 3 points to Query



IVF: Query time

Ex. Find closest 3 points to Query

Pick the nearest cluster to query and search there?

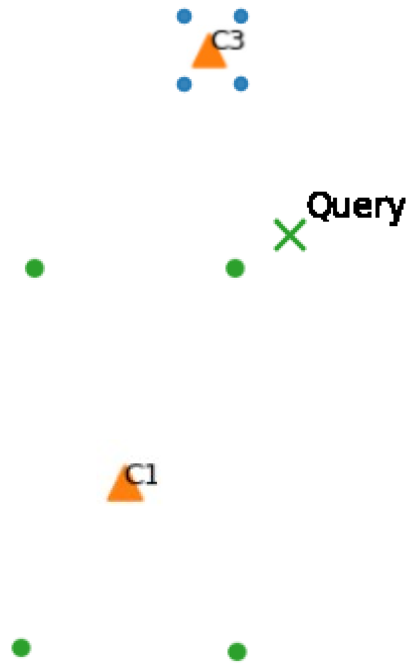


IVF: Query time

Ex. Find closest 3 points to Query

Pick the nearest cluster to query and search there?

What if you have this situation?



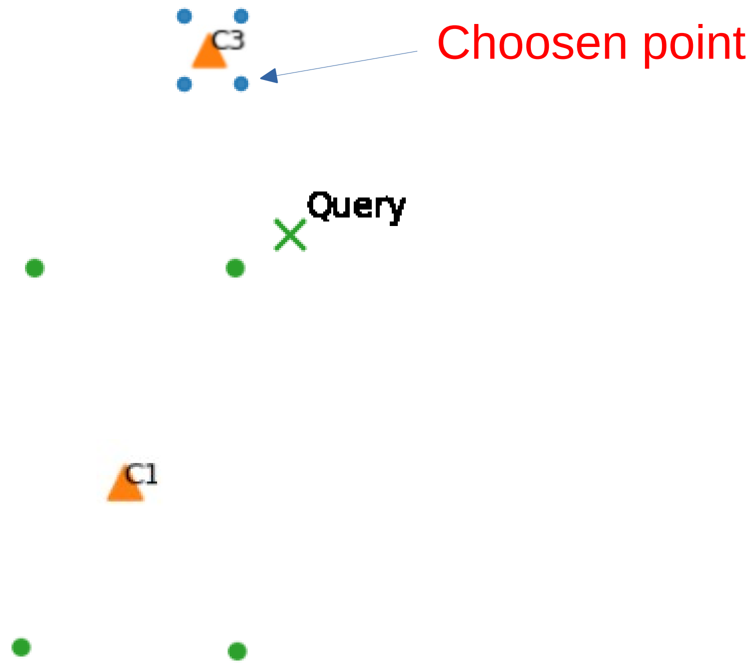
IVF: Query time

Ex. Find closest 3 points to Query

Pick the nearest cluster to query and search there?

What if you have this situation?

Query is closest to C3, so lower right blue point is marked as closest



IVF: Query time

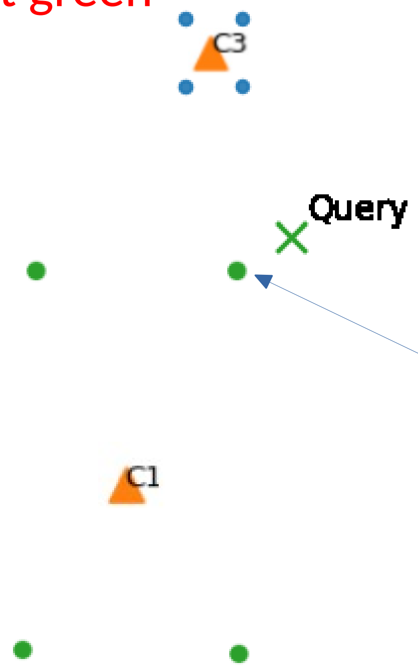
Ex. Find closest 3 points to Query

Pick the nearest cluster to query and search there?

What if you have this situation?

Query is closest to C3, so lower right blue point is marked as closest

Correct is upper right green



Choosen point

IVF: Query time

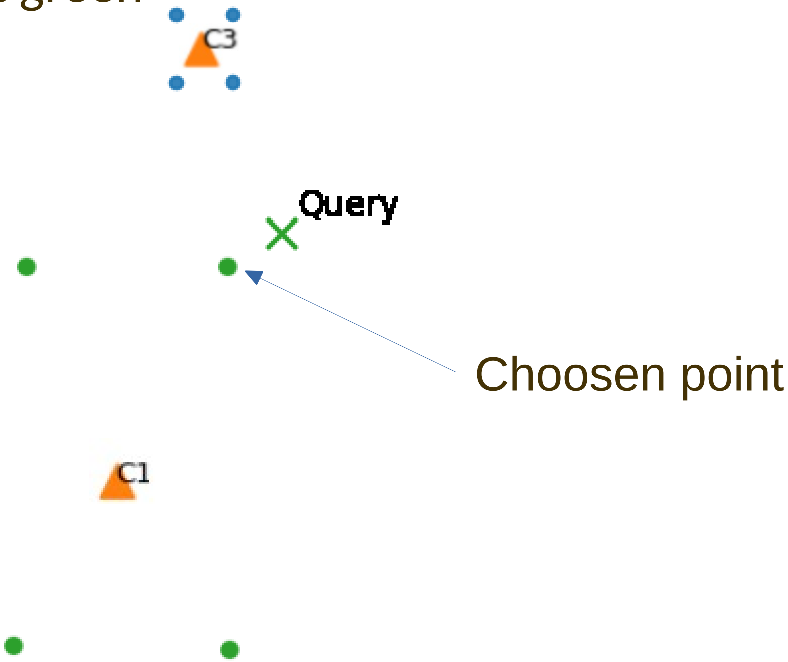
Ex. Find closest 3 points to Query

Pick the nearest cluster to query and search there?

What if you have this situation?

Query is closest to C3, so lower right blue point is marked as closest

Correct is upper right green

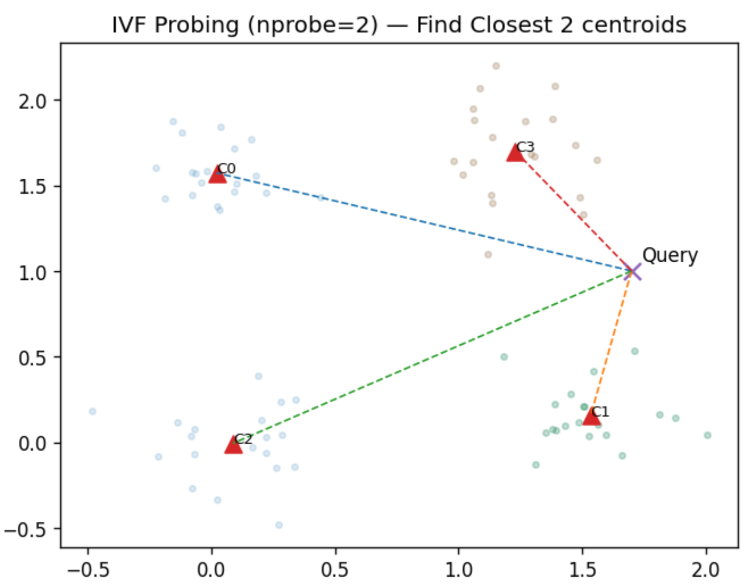


Partial solution is to allow searching more than 1 cluster

IVF: Query time

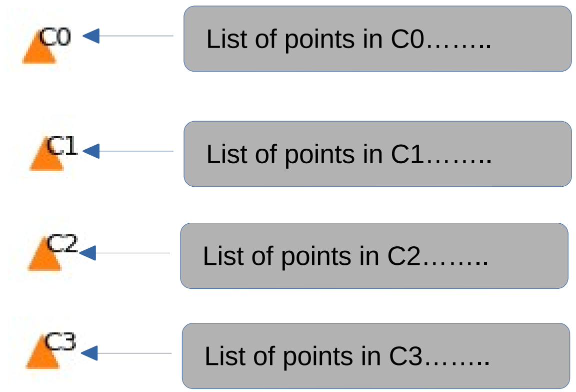
Ex. Find closest 3 points to Query

Find Closest **2** clusters



C1 and C3 are closest centroids

Use the Lists for C1 and C3 to get all the points to compare to

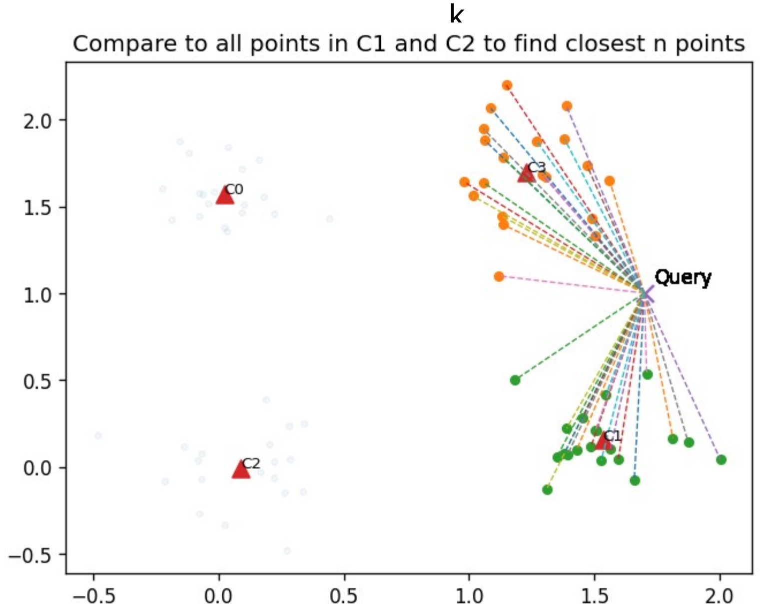


IVF: Query time

Ex. Find closest 3 points to Query

Find Closest 2 clusters

Calculate distances to all points in each cluster



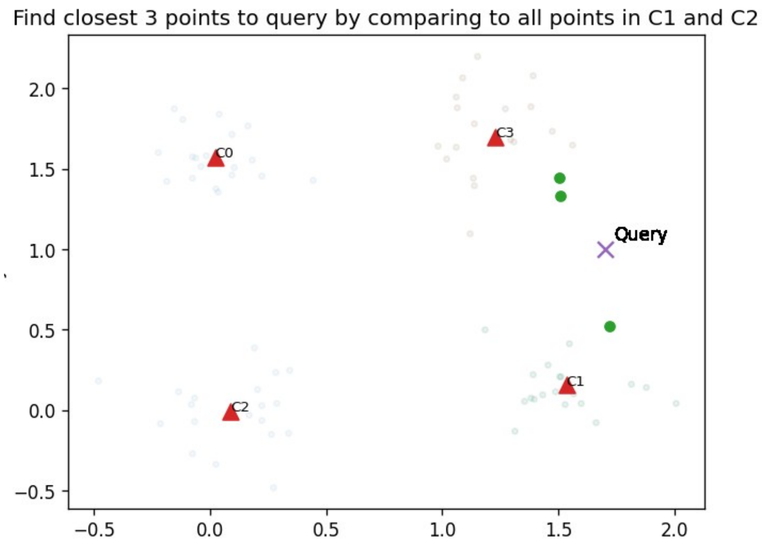
IVF: Query time

Ex. Find closest 3 points to Query

Find Closest 2 clusters

Calculate distances to all points in each cluster

Sort distances,Select closest 3 points



HNSW: Small-World Graph — Idea

Multi-layer graph; greedy search from top layer, descending to denser layers.

Layer L (sparse)

Layer L-1

Layer L-2 (dense)

HNSW: Mechanics & Tuning

- Build: M = max neighbors; efConstruction controls build quality/time
- Query: efSearch controls recall vs latency
- Trade-offs: ↑M, ↑efSearch → ↑recall & memory/latency

Quick Comparison

Aspect	kNN (Exact)	IVF	HNSW
Query time	Highest (O(n))	Low, tunable (nprobe - #clusters)	Very low, tunable (efSearch)
Build time	None	Moderate (k-means)	High (graph build)
Memory	Data only	Data + centroids	Data + graph links (higher)

- kNN will find exact match
- IVF and HNSW may not

Summary

- kNN is exact but slow for large n
- IVF: partition + probe a few lists (nlist, nprobe)
- HNSW: layered graph, fast greedy search (M, efSearch)